If the condition remains true, the following code bracketed by the curly braces will be executed. If, however, the condition returns false, the `for` loop will end. The third and final statement increments the counter variable each time the enclosed block of code is executed.

Let's say we wanted to blink an LED five times quickly, we could use a `for` loop similar to the following:

```
for (int i = 0; i < 5; i++) {
  digitalWrite(13, HIGH);
  delay(250);
  digitalWrite(13, LOW);
  delay(250);
}
```

In this sample code, the first time through the `for` loop, we declare an integer type variable to serve as our index or counter and assign it the value 0 in this statement:

```
int i = 0;
```

Each time through the loop, the conditional statement will be checked to see if the condition remains true. Take, for example, the following:

```
i < 5;
```

In this statement as long as `i` is less than 5, the enclosed statements will be executed, turning on pin 13, waiting for 250 milliseconds, turning pin 13 off, then waiting another 250 milliseconds.

Each time through the loop, after all of the statements within the curly braces have been executed, the variable `i` is incremented, in an increment statement:

```
i++;
```

In this case we add 1 to the value of `i` and reassign this new value back to `i` so that each time through the loop `i` increases by 1. Remember the compound operator `i++` is functionally identical to `i = i + 1`. In this way, `i` starts at 0 the first time through the loop, incrementing 5 times each time through the loop until its value is no longer less than 5, consequently ending the loop. In our project code, an example of the `for` loop in action includes the following example:

```
for (int i=0; i<3; i++) pinMode(rgb[i], OUTPUT);
```

Here, we are using a local variable `i` declared inside the loop to be used as a counter and assigned the value 0. For as long as `i` remains less than 3, expressed in the conditional statement `i<3;`, the code following the `for` statement will be executed. The for loop will then repeat three times, and each time the `pinMode()` function will be executed, setting a pin, as defined by the `rgb[]` array, as an output. Because we only needed a single simple statement to be executed, the curly braces are not required.

You can also see that we have used the counter `i` to increment the position of the array `rgb[]` being referenced. After that statement has been executed, the counter `i` will be incremented by one. Once `i` has reached the value `3`, the `for` loop will terminate and proceed with the standard program flow to the following line of code.

While

The `for` statement is fairly common to Arduino programmers, but there are other ways to structure iterative loops. Where the `if` statement executed a statement once if a condition was met, and the `for` loop cycles through a specified number of times, the `while` statement, or `while` loop, is used to